FASE 3: DEV pleno

FLASHCARDS DE MÉTRICAS DE PROCESSO

ASSUNTO 1: Introdução e Visão Holística das Métricas de Processo

Flashcard 1.1 - Definição e Propósito: As Métricas de Processo avaliam como o software é desenvolvido e evolui ao longo do tempo. Diferente das métricas de produto (que analisam a estrutura estática do código-fonte), elas focam na dinâmica, no fluxo de trabalho e na eficácia da equipe de engenharia.

Flashcard 1.2 - Por que é Importante / Vantagens: Ajudam a responder três perguntas vitais sobre a saúde do projeto: Eficiência (o time está entregando no ritmo esperado?), Qualidade (o produto atende aos requisitos com estabilidade?) e Esforço & Correção (quanto tempo é gasto apagando incêndios e refazendo tarefas?). São indispensáveis para a gestão e melhoria contínua dos processos.

Flashcard 1.3 - Interdependência das Métricas (Visão Holística): Nenhuma métrica de processo deve ser analisada de forma isolada! Por exemplo, uma velocidade de entrega enganosa (alta produtividade) pode esconder uma queda crítica de qualidade (alta densidade de defeitos). O aumento de bugs gera um efeito dominó de refação imediata, que por sua vez cria um gargalo de retrabalho e derruba a produtividade efetiva.

Flashcard 1.4 - Exemplo Prático (Visão Holística): Se uma equipe comemora a entrega de 40 histórias em uma sprint (alta produtividade), mas gasta 80% do tempo da sprint seguinte corrigindo bugs que vazaram dessas entregas, a análise holística revela que a ”velocidade de entrega” inicial foi falsa e prejudicial à saúde do projeto.

ASSUNTO 2: Produtividade de Software

Flashcard 2.1 - Definição e Propósito: A Produtividade mede a quantidade de software produzido pela equipe por unidade de esforço ou tempo investido. Ela serve como um termômetro de eficiência para entender se os recursos do projeto estão sendo bem utilizados.

Flashcard 2.2 - Por que é Importante / Vantagens: Ajuda a mapear a capacidade de entrega real da equipe. Sabendo a produtividade média, os gestores conseguem planejar cronogramas, estimar prazos de sprints futuras com maior precisão e detectar quedas de ritmo causadas por bloqueios técnicos ou organizacionais.

Flashcard 2.3 - Como Calcula / Fórmulas: A produtividade é calculada dividindo-se o volume total de saídas funcionais geradas (outputs) pelo total de insumos ou esforço investido (inputs):

Produtividade = Saída (Output) / Entrada (Input)

As unidades mais comuns são: Linhas de Código (LOC) por dev/mês, Pontos de Função (FP) por sprint, ou Histórias de Usuário entregues por hora de esforço.

Flashcard 2.4 - Exemplo Prático de Cálculo: Um time de 5 pessoas trabalha durante 2 semanas ($8h$/dia de segunda a sexta, totalizando 10 dias de trabalho). O esforço total do período

é $5 \times 10 \times 8 = 400$ horas. Se eles entregaram exatamente 20 histórias de usuário, a produtividade em histórias por hora de esforço é:

$$\text{Produtividade} = \frac{20 \text{ histórias}}{400 \text{ horas}} = \mathbf{0.05} \text{ histórias/hora}$$

ASSUNTO 3: Densidade de Defeitos e Taxa de Escape

Flashcard 3.1 - Definição e Propósito: A Densidade de Defeitos mede a quantidade de bugs identificados no software de forma normalizada pelo tamanho do produto. Essa normalização é essencial para permitir comparações justas de qualidade entre módulos de tamanhos diferentes.

Flashcard 3.2 - Por que é Importante / Vantagens: Uma densidade alta indica que o código daquele módulo está instável ou é excessivamente complexo. Já uma densidade muito baixa exige atenção cautelosa: ela pode indicar um software altamente estável, ou simplesmente revelar que a estratégia de testes da equipe é ineficiente e não está encontrando os bugs existentes.

Flashcard 3.3 - Como Calcula / Fórmulas: O cálculo consiste em dividir o número total de defeitos registrados pelo tamanho do software (geralmente medido em Milhares de Linhas de Código - KLOC ou em Pontos de Função - FP):

$$\text{Densidade de Defeitos} = \frac{\text{Nº de Defeitos}}{\text{Tamanho (KLOC ou FP)}}$$

A equipe também deve monitorar a Taxa de Escape, que mede a proporção de defeitos que não foram capturados pelas baterias de testes internos e vazaram diretamente para os usuários finais em produção.

Flashcard 3.4 - Exemplo Prático de Cálculo: Durante as duas semanas de testes de um novo módulo estimado em 70 Pontos de Função (FP), a equipe de controle de qualidade registrou e documentou 14 defeitos. A densidade de defeitos do módulo é calculada da seguinte forma:

$$\text{Densidade de Defeitos} = \frac{14 \text{ defeitos}}{70 \text{ FP}} = \mathbf{0.2} \text{ defeitos/FP}$$

ASSUNTO 4: Taxas de Retrabalho (Rework Rates)

Flashcard 4.1 - Definição e Propósito: O Retrabalho mede a quantidade de esforço ou tempo gasto para refazer, corrigir ou alterar tarefas após a entrega inicial devido a falhas de comunicação, codificação incorreta ou refinamento inadequado. É a métrica oficial do desperdício de tempo e recursos do projeto.

Flashcard 4.2 - Por que é Importante / Vantagens: Uma alta taxa de retrabalho drena a capacidade de inovação e desenvolvimento da equipe, uma vez que o time passa a gastar a maior parte do seu tempo corrigindo coisas que já deveriam estar prontas. Indica gargalos severos no fluxo inicial de design de código, falta de testes automatizados ou requisitos mal definidos.

Flashcard 4.3 - Como Calcula / Fórmulas: Pode ser medido rastreando-se o número de commits de correção em features já fechadas, PRs rejeitados em revisão, histórias reabertas na

sprint, ou de forma percentual com base no esforço total (horas gastas em refação divididas pelo esforço total disponível no período):

$$\text{Taxa de Retrabalho (Esforço)} = \frac{\text{Horas de Correção} + \text{Horas de Requisitos Refeitos}}{\text{Esforço Total do Período}} \times 100$$

Flashcard 4.4 - Exemplo Prático de Cálculo: Em um ciclo com esforço total disponível de 400 horas de trabalho, o time gastou 60 horas corrigindo defeitos de código e outras 40 horas reescrevendo e adaptando requisitos que haviam sido mal interpretados pelo cliente. A taxa de retrabalho por esforço do período é:

$$\text{Taxa de Retrabalho} = \frac{60h + 40h}{400h} \times 100 = \frac{100h}{400h} \times 100 = \textbf{25\%}$$

ASSUNTO 5: O Custo do Retrabalho no Tempo (Custo Exponencial)

Flashcard 5.1 - Definição e Propósito: Avalia o impacto financeiro, temporal e de imagem sofrido pelo projeto dependendo de qual fase do ciclo de desenvolvimento um bug é encontrado. Baseia-se no princípio de que quanto mais tarde uma falha é detectada, exponencialmente mais caro se torna o seu conserto.

Flashcard 5.2 - Por que é Importante / Vantagens: Justifica a aplicação de práticas de "Shift-Left" (antecipar testes e validações para as fases iniciais do projeto). Provar para a equipe que a prevenção de bugs na fase de requisitos custa uma fração minúscula do custo de correção de bugs em produção ajuda a elevar a maturidade do processo de engenharia.

Flashcard 5.3 - O Custo Exponencial (Escalas): O custo de correção cresce de forma assustadora entre as fases:

- **Fase de Requisitos (Custo 1x):** Extremamente barato. Basta reescrever ou ajustar um trecho do documento de especificação.
- **Fase de Codificação (Custo 10x):** O desenvolvedor precisa interromper sua tarefa atual para reescrever e recompilar a lógica localmente.
- **Fase de QA / Testes (Custo 100x):** Todo o ciclo de build, empacotamento, deploy em homologação e tempo dos analistas de testes é perdido e precisa ser refeito.
- **Fase de Produção (Custo 1000x):** Impacta o usuário final. Exige hotfixes emergenciais, sobrecarrega a equipe, gera chamados de suporte e causa danos sérios à imagem e reputação da marca.

Flashcard 5.4 - Exemplo Prático: Um bug de cálculo de imposto pego durante a revisão de requisitos custa apenas alguns minutos de conversa (1x). O mesmo bug, se descoberto meses depois em produção após milhares de notas fiscais terem sido emitidas incorretamente, exigirá paralisar o faturamento, criar código emergencial às pressas, cancelar e reemitir notas, gerando custos de multas tributárias e reclamações públicas de clientes (1000x).